SOFTWARE RELEASE NOTE
& TEST EVIDENCE REPORT

Rapid Tech Insights — Application Fixes, Validation and Release Readiness

Rapid Inventory (supermarket inventory and order-management system)

Document control	
Document title	Software Release Note & Test Evidence Report
Document ID	RTI-SRN-RINV-2026-001 [proposed — to be confirmed]
Document version	1.0
Preparation date	29 October 2026
Prepared by	alexkemboi (Git author of the release commits), with AI assistance (Claude Code) — [full name to be confirmed]
Reviewed by	[To be confirmed]
Approved by	[Pending — not approved]
Application / project	Rapid Inventory — repository github.com/alexkemboi/Supermarket-Inventory-System
Application version	1.1.0 [proposed — the repository defines no version]
Release number	1.1.0.291020260602 [proposed; template format Major.minor.patch.ddmmyyyyhhmn, from release commit 4555d20]
Environment	Development / test — local Docker Compose stack only. No staging or production deployment performed.
Release status	Release candidate — pending QA review and approval
Document classification	Internal — Confidential [proposed, to be confirmed]

Table of Contents

1. Document Version Control **3**
2. Product Release Note **4**
2.1 General Release Information 4
2.2 Purpose of Release 4
2.3 Release Overview 5
2.4 Modules and Functions Affected 5
2.5 Requirement 6
2.6 Fix Release Details 7
3. Root Cause and Fix Summary **8**
3.1 INV-001 — Inventory Overselling 8
3.2 INV-002 — Slow Inventory Listing 9
3.3 INV-003 — SignalR Stock Synchronisation 9
3.4 INV-004 — Order Placement Refactoring 10
3.5 INV-005 — Low-Stock Alerts 10
4. Test Execution Summary **12**
4.1 Test Summary Table 12
4.2 Test Result Classification 15
5. Screenshots and Test Evidence **16**
5.1 Evidence A — Application Startup 16
5.2 Evidence B — Inventory Concurrency (INV-001) 18
5.3 Evidence C — Inventory Performance (INV-002) 22
5.4 Evidence D — SignalR Synchronisation (INV-003) 25
5.5 Evidence E — Order Placement Refactoring (INV-004) 32
5.6 Evidence F — Low-Stock Notifications (INV-005) 34
5.7 Evidence G — Final Validation 41
5.8 Evidence Traceability Matrix 43
6. Installation Guide and Environment Setup **44**
6.1 Prerequisites 44
6.2 Configuration and Environment Variables 44
6.3 Installation Steps 44
6.4 Post-deployment Verification (Test Instruction) 45
7. Release Prerequisites **46**
8. Rollback Procedure **47**
9. Known Issues and Residual Risks **48**
10. Release Acceptance and Sign-off **49**

Field codes: if page numbers show as unrefreshed, right-click the table and choose Update Field → Update entire table.

1. Document Version Control

Date	Version	Author	Change History	Reviewer / Approver
29 Oct 2026	1.0	alexkemboi [full name to be confirmed], AI-assisted (Claude Code)	Initial release note and test evidence report for Rapid Inventory release 1.1.0 (proposed). Rebranded from the System Release Note template v1.0.1 to Rapid Tech Insights.	[To be confirmed] / [Pending]

Source template: `SystemReleaseNote.Template.1.0.1.170720241030.docx`. Its sections (version control, contact details, general release information, fix release details with CRC check, installation guide and environment setup, change request, purpose, requirement, modules affected, test instruction, prerequisite, rollback process, test evidence) are retained below; the original company branding and contact details have been replaced by Rapid Tech Insights details verified on rapidtechinsights.com.

2. Product Release Note

2.1 General Release Information

Contact details	
Developer / release owner	alexkemboi (Git author, all release commits) — [full name and role to be confirmed]
Support email	[To be confirmed — no application support mailbox supplied]. Company contact (verified on website): projects@rapidtechinsights.com
Support phone no.	[To be confirmed]. Company line (verified on website): +254 112 130 196
Release maker	[To be confirmed]
Release checker / QA	[To be confirmed]
General release info	
Application name	Rapid Inventory — supermarket inventory and order-management system (ASP.NET Core API, Angular web app, SQL Server, SignalR)
Application version	1.1.0 [proposed; no version is defined in the repository — frontend package.json reports 0.0.0]
Release date	[To be confirmed] — code complete 29 Oct 2026 06:02 EAT (commit 4555d20)
Release number	1.1.0.291020260602 [proposed]. Format (Major.minor.patch.ddmmyyyyhhmn) retained from template.
Release branch	`main` — origin https://github.com/alexkemboi/Supermarket-Inventory-System (HEAD 4555d20 at time of testing)
Client / business unit	[To be confirmed]
Deployment environment	Verified only in a local development/test environment (Docker Compose: api, web, db). Staging/production: not performed.
Change request / issue reference	Stage 3 practical assessment brief (reported issues INV-001 … INV-005 in this document). CR / CRM case number: [To be confirmed]
Release type	Corrective (defect fixes) + performance + minor enhancement (low-stock toasts) + internal refactoring
Release status	Release candidate — engineering verification complete in test environment; QA sign-off and approval pending

2.2 Purpose of Release

This release addresses five issues reported against the supermarket inventory system. They were treated as reported symptoms until the repository, reproduction tests and runtime evidence established their causes (Section 3):

- **INV-001** Concurrent customer orders can result in more stock being ordered than is available — risk of selling goods that do not exist, inaccurate inventory and customer disappointment.

- **INV-002** The inventory listing page responds slowly — staff wait on every search and refresh.
- **INV-003** The live stock board does not update for other connected users — staff act on stale stock figures.
- **INV-004** Order placement logic has accumulated business responsibilities inside the controller — harder to change safely and to test.
- **INV-005** Users need low-stock notifications when inventory falls below the reorder threshold.

Intended improvements: guaranteed inventory integrity under concurrent load across API instances, a responsive inventory page, real-time stock updates for every connected user, a maintainable order workflow, and visible low-stock alerts.

2.3 Release Overview

Changes listed here are included only where the source code and executed verification support them.

Area	Change implemented	Commit
Inventory concurrency	Single conditional SQL `UPDATE … WHERE Stock >= @qty` inside a transaction that also inserts the order; `CK_Products_Stock_NonNegative` check constraint; notifications only after commit.	6157517
Query optimisation	Inventory list projected to `ProductListDto` with a join: 202 → 2 SQL commands per page. No index or cache added (not justified by the plan).	2f076ee
SignalR synchronisation	`StockUpdated` always broadcast to the `stock-watchers` group; Angular client restarts a closed connection with backoff.	d8329cc
Order service refactoring	`OrderService` owns place / cancel / restock; controller maps results to HTTP. Cancel made race-safe.	75569d9
Low-stock alerts	Global, dismissable toast in the Angular shell for `LowStockAlert`; per-alert ids; listener lifecycle tested.	1bfb08b
Regression testing	xUnit integration tests on SQL Server (42), Karma specs (17), headless two-browser check, load generator.	6bae192, 6028cbd, 280ac2d
Documentation	`ROOT_CAUSE_ANALYSIS.md`, `SOLUTIONS.md`, walkthrough script, README test section.	4555d20
Database migration	`20261029024124_AddNonNegativeStockConstraint` (adds a CHECK constraint only; no data change).	6157517

2.4 Modules and Functions Affected

Module / Function	Previous behaviour	Change implemented	Expected or verified outcome
`POST /api/orders` — backend/Services/OrderService.cs (PlaceOrderAsync)	Read stock, then absolute `SET Stock = @newStock`; no transaction; 500 if SignalR failed after save	Conditional atomic deduction inside a transaction; post-commit best-effort notifications	**Verified** — TC-01…TC-14 PASS

Module / Function	Previous behaviour	Change implemented	Expected or verified outcome
`POST /api/orders/{id}/cancel` — OrderService.CancelOrderAsync	Read-check-write on status; notify before save; 6 of 10 concurrent cancels succeeded	Conditional status `UPDATE` in a transaction; publish after commit	**Verified** — TC-12, TC-43 PASS
`POST /api/products/{id}/restock` — OrderService.RestockAsync	Broadcast a value computed from a pre-read	Relative update + read in transaction; publish committed value	**Verified** — TC-34, TC-43 PASS
`OrdersController` (backend/Controllers)	Contained all order business logic	Thin adapter: `ToActionResult(ServiceResult)`	**Verified** — 13 contract tests identical before/after
`GET /api/products` — ProductsController.List	N+1: 202 SQL commands for 200 rows; tracked entities	DTO projection with join; 2 commands	**Verified** — median 224.9 → 11.9 ms
`POST/PUT /api/products` — ProductsController	Accepted negative stock	400 "Stock cannot be negative."	Expected (guard for new constraint); not separately tested
Products table (SQL Server)	No integrity constraint on stock	`CHECK ([Stock] >= 0)` via EF migration	**Verified** — TC-11 PASS; EV-A2
`SignalRStockNotifier` (backend/Services/IStockNotifier.cs)	Sent `StockUpdated` only to the requester when a connection id was supplied	Always sends to group `stock-watchers`; alerts use `LowStockAlertDto`	**Verified** — TC-30…TC-35 PASS
`StockRealtimeService` (frontend)	Gave up after automatic reconnect failed; alerts keyed by product	Bounded restart + refresh; per-alert ids; dismiss; injectable connection	**Verified** by unit specs; live outage test NOT RUN
`LowStockAlerts` component + app shell (frontend)	Alerts only on Live board page	Toast region on every page (aria-live)	**Verified** — TC-52…TC-58 PASS
`board.html` (frontend)	`track alert.productId` (duplicate keys on repeat alerts)	`track alert.id`	**Verified** — Karma specs

2.5 Requirement

- A successful order must never consume more stock than exists, regardless of how many API instances process orders concurrently; stock must never be negative.
- A failed order (validation, not found, insufficient stock, or any error) must leave no order records and no stock deductions.
- The inventory list must return the same data and contract while avoiding per-row database queries.
- Stock changes must reach every connected client of the API instance; failed orders must not publish updates.
- A `LowStockAlert` is raised only by an order that moves stock from at-or-above the reorder threshold to strictly below it, and is shown to users in the Angular application.

- Existing request/response contracts and error semantics must be preserved. [Business Requirements Document update: to be confirmed]

2.6 Fix Release Details

Source filename	`release/package/rapid-inventory-1.1.0-src-4555d20.zip` (git archive of commit 4555d20)
CRC check (CRC32)	**CE4AA374**
SHA-256	`18f69d171e2ad2345679f22a152fe1c7081d82d2c29912b6bbe10c82a512d503`
Size / entries	157,850 bytes / 95 entries
Package modified	29 Oct 2026 06:19:33 (local, EAT)
Evidence	`Release_Evidence/logs/EV-G6_release_package_checksums.txt` (values computed, not a screenshot)

The template asks for a screenshot of the zip CRC32 and modification date; the values above were computed from the archive with Python `zlib.crc32` / `hashlib.sha256` and are recorded as a text artifact. An approved, signed release artifact has not yet been produced.

3. Root Cause and Fix Summary

Issue ID	Reported symptom	Confirmed root cause	Corrective action	Verification status
INV-001	Concurrent orders can oversell stock	Read → in-memory check → absolute `SET Stock = @newStock` in a separate auto-commit (lost update); no transaction around deductions and order insert	Conditional atomic `UPDATE … WHERE Stock >= @qty` in one transaction with the order insert; CHECK constraint; post-commit notifications	**PASS** Resolved in test env — TC-01…TC-14
INV-002	Inventory list slow	N+1 queries: one `Categories` query per product row (202 commands per 200-row page); entity materialisation and tracking	Projection to DTO with join (2 commands); no index needed	**PASS** Resolved in test env — TC-20…TC-25
INV-003	Live board not updating for other users	`StockUpdated` sent to `Clients.Client(requester)` whenever a connection id was supplied — always the case from the inventory page	Broadcast to `stock-watchers` group; client restart after reconnect gives up	**PASS** Resolved for single-instance deployment — TC-30…TC-35; reconnection PARTIAL; multi-instance NOT RUN
INV-004	Business logic in controller	Controller owned validation, stock logic, persistence, transactions, notifications; unused stale `OrderService` duplicated it; cancel race	`OrderService` owns use cases; thin controller; transactional cancel/restock	**PASS** Behaviour preserved — TC-40…TC-45
INV-005	Low-stock notifications needed	Backend already emitted `LowStockAlert`; UI rendered it only on the Live board and keyed it by product id; stale reads made the transition unreliable	Global toast; per-alert ids; exact transition from transactional before/after values	**PASS** Resolved in test env — TC-50…TC-59

3.1 INV-001 — Inventory Overselling

Confirmed cause. The original `OrdersController.Place` (commit fc3c873) selected the product, checked `Stock < Quantity` in memory and then wrote an absolute value. SQL captured from the original API (EV-B5):

```
SELECT TOP(2) [p].[Id], [p].[Name], [p].[UnitPrice], [p].[Stock], [p].[ReorderThreshold] FROM [Products] AS [p] WHERE
[p].[Id] = @id
UPDATE [p] SET [p].[Stock] = @__newStock_1 FROM [Products] AS [p] WHERE [p].[Id] = @__line_ProductId_0
INSERT INTO [Orders] ... ; INSERT INTO [OrderLines] ...   -- separate SaveChanges transaction
```

Two requests can both read the same stock, both pass the check and both write the same new value. Stock is not driven negative; instead more units are recorded in order lines than were deducted (baseline: 40 units sold from stock 10 with final stock 5 — EV-B1; stress script: 40 units sold while stock fell 30 → 13 — EV-B0). Deductions were auto-committed before the order insert, so a failure on a later line left earlier deductions committed.

Fix. `OrderService.PlaceOrderAsync` opens a transaction and, per line, executes the check and the deduction as one statement. SQL captured from the fixed API (EV-B6):

```
UPDATE [p] SET [p].[Stock] = [p].[Stock] - @__line_Quantity_1
FROM [Products] AS [p]
WHERE [p].[Id] = @__line_ProductId_0 AND [p].[Stock] >= @__line_Quantity_1
```

SQL Server must take the row's update lock before evaluating `Stock >= @qty`, so competing requests from any API instance serialise on the row; a request that obtains the lock after the last unit has gone re-evaluates the predicate, affects 0 rows and returns **409 Insufficient stock** with a full rollback. No in-process lock is involved, so the guarantee holds across instances. `CK_Products_Stock_NonNegative` rejects any negative value at the database. References: `backend/Services/OrderService.cs`, `backend/Data/AppDbContext.cs`, `tests/Inventory.Api.Tests/OrderConcurrencyTests.cs`.

3.2 INV-002 — Slow Inventory Listing

Confirmed cause. The Angular page requests `pageSize=200`. `ProductsController.List` loaded tracked `Product` entities, then ran one category lookup per row: 1 count + 1 page query + 200 × `SELECT TOP(2) [c].[Name] FROM [Categories] AS [c] WHERE [c].[Id] = @id` = **202 commands** (EV-C1). The 250,000-row Orders table is not used by this endpoint, so the seed volume was not the cause.

Fix. One projected query with an `INNER JOIN` to Categories, selecting only DTO columns (EV-C2, EV-C7). Execution plan: ordered clustered-index scan of `PK_Products` with `TOP(200)` (no sort) and nested-loop seeks on `PK_Categories`; 6 logical reads on Products (EV-C5, EV-C6). No index was added: paging already uses the PK and search is a leading-wildcard `LIKE` that no B-tree index can seek.

Measurement (50 sequential requests after 5 warm-ups, API restarted before each series)	Before	After
`GET /api/products?pageSize=200` (inventory page) — median / p95	224.9 ms / 261.0 ms	**11.9 ms / 15.5 ms**
`…&search=Milk` — median / p95	95.4 ms / 114.7 ms	22.8 ms / 29.9 ms
`?page=3&pageSize=50` — median / p95	68.6 ms / 88.7 ms	7.2 ms / 8.7 ms
SQL commands per page request	202	2
Response payload (pageSize=200)	28,978 bytes	28,978 bytes (unchanged)

Both series ran on the same seeded database under identical conditions (Development mode with SQL command logging enabled, which adds overhead to both).

3.3 INV-003 — SignalR Stock Synchronisation

Event flow traced. Order committed → `IStockNotifier.StockUpdatedAsync` → hub `/hubs/stock` → clients that invoked `JoinStockWatchers` (group `stock-watchers`) → Angular `StockRealtimeService` handler `StockUpdated(productId, stock)` → `stockUpdated` subject → `Inventory`/`Board` components patch the row. Hub route, client URL, proxy (`ws: true`), event names, argument shape and group join all matched.

Confirmed cause. In `SignalRStockNotifier`, `requesterConnectionId is null ? Clients.Group(...) : Clients.Client(requesterConnectionId)`. The inventory page always sends its own connection id with an

order, so the update went only to the ordering user. Restock sends no id — which is why restocks did reach everyone. Baseline browser check: Browser B stayed at 41 while the database held 40 (EV-D0).

Fix. Always send to the group (EV-D8); the requester applies the same value idempotently. The client now restarts a closed connection with capped backoff and reloads data once live. Updates are published only after commit, so failed orders publish nothing (EV-D6, EV-F5). **Limitation:** without a SignalR backplane, clients only receive events from the API instance they are connected to.

3.4 INV-004 — Order Placement Refactoring

Responsibility	Before (OrdersController)	After
Validation (empty order, quantity ≤ 0)	Controller	`OrderService.ValidateOrder`
Stock check and deduction	Controller (read → check → absolute write)	`OrderService` (conditional UPDATE in transaction)
Order construction, persistence, transaction	Controller (no transaction)	`OrderService`
Cancel / restock	Controller (no transaction, publish before save / stale value)	`CancelOrderAsync` (conditional status claim) / `RestockAsync`
Low-stock evaluation	Controller (inline)	`OrderService.IsLowStockTransition`
SignalR publication	Controller	`OrderService.PublishAsync` (post-commit, best effort, logged)
HTTP mapping	Mixed with business logic	`OrdersController.ToActionResult(ServiceResult)`

No new interfaces, repositories or layers were added — `OrderService`, `ServiceResult` and `IStockNotifier` already existed. Behaviour preservation is shown by 13 contract tests (status codes, `{ error }` messages, `{ id }` body, cancel/restock semantics) that passed on the original controller (EV-B1) and pass unchanged after the refactor (EV-E3, EV-G3). Intentional behaviour changes are limited to the defect fixes, each covered by a test that failed before.

3.5 INV-005 — Low-Stock Alerts

Threshold field	`Product.ReorderThreshold` (existing; seeded 15). No new field introduced.
Condition	Alert only when the order moves stock from ≥ threshold to < threshold (previous ≥ T and current < T). Exactly at T: no alert. Already below: no repeat. Restock to ≥ T re-arms.
When evaluated	Inside the order transaction from exact before/after values read under the row lock; published after commit.
SignalR event	`LowStockAlert` to group `stock-watchers`

Payload (`LowStockAlertDto`)	`{ productId, productName, stock, reorderThreshold }` — captured frame: `{"productId":41,"productName":"Sukuma Wiki 500g","stock":14,"reorderThreshold":15}` (EV-F5)
Angular UI	`LowStockAlerts` toast region in the app shell (every page), `aria-live="polite"`, dismiss button; max 5 alerts; also listed on the Live board.
Failed transactions	No alert and no stock update are published (TC-55, TC-56; EV-D6).
Delivery limitations	At-most-once: if publishing fails after commit the alert is lost (logged warning, no outbox); disconnected clients and later page loads do not see past alerts; multi-instance needs a backplane.

4. Test Execution Summary

Scope	INV-001 … INV-005: order concurrency and integrity, inventory list performance and correctness, SignalR delivery, refactoring regression, low-stock alerts, builds.
Environment	Windows 11 host (8 CPU, 16 GB); Docker 29.8.2 / Compose v5.6.0; SQL Server 2022 16.0.4305.5 container; API .NET 9 (SDK 9.0.320, ASP.NET Core 9.0.22, EF Core 9.0.10, Development mode); Angular 20.3.34, @microsoft/signalr 10.0.11; Chrome 156 headless; Node 24.21.0.
Execution date	29 October 2026 (EAT, UTC+3). Evidence captured 02:30–03:23 UTC.
Approach	Reproduce first on the original code (commit fc3c873), then fix and re-run. Integration tests run against real SQL Server (disposable database per test class), using two independent in-process API hosts for multi-instance cases; live checks against the Docker stack; real browsers via headless Chrome DevTools Protocol; a second API process for multi-process load.
Frameworks / commands	`dotnet test Inventory.sln` (xUnit 2.9.3, Microsoft.AspNetCore.Mvc.Testing 9.0.10) · `npx ng test --watch=false --browsers=ChromeHeadless` (Karma/Jasmine) · `npm run build` · `node seed/two-browser-check.mjs` · `node seed/load-test.mjs` · `./seed/stress-orders.sh`
Overall result	Backend 42/42 passed (baseline: 31 passed, 9 failed — the 9 reported defects). Frontend 17/17 passed. Backend build 0 warnings / 0 errors. Frontend production build succeeded. Not run: human-operated two-browser test, live reconnection outage, staging deployment, Release-build load test, multi-instance SignalR, deadlock scenario.

4.1 Test Summary Table

Test ID	Test scenario	Expected result	Actual result	Status	Evidence
Inventory concurrency					
TC-01	Order less than available stock	200; stock reduced by qty	200; 10 → 7	**PASS**	EV-G3a
TC-02	Order exceeding available stock	409; no stock/order change	409 "Insufficient stock…"; stock 4, 0 lines	**PASS**	EV-G3a
TC-03	Order exactly the available stock	200; stock 0	200; stock 0	**PASS**	EV-G3a
TC-04	Two API instances race for the last unit (deterministic, lock-forced)	One 200, one 409; stock 0; 1 order line	Fixed: OK, Conflict; stock 0. Baseline: OK, OK	**PASS**	EV-B2, EV-B1
TC-05	40 concurrent orders across 2 instances, stock 10	10 OK, 30 Conflict, stock 0, 10 units	10 OK, 30 Conflict, stock 0, 10 units. Baseline: 40 OK, stock 5	**PASS**	EV-B2, EV-B1
TC-06	30 concurrent mixed-quantity orders, stock 25	Sold units = stock deducted; no reject that would fit	Consistent; baseline 73 units sold from 25	**PASS**	EV-G3a
TC-07	Later line insufficient → rollback earlier line	409; earlier product stock unchanged; no order	409; stock 5 unchanged; 0 orders without lines	**PASS**	EV-G3a

Test ID	Test scenario	Expected result	Actual result	Status	Evidence
TC-08	Later line unknown product → rollback	404; stock unchanged	404; stock 5 unchanged	PASS	EV-G3a
TC-09	SignalR failure after commit (duplicate-submission risk)	200; exactly one order and deduction	200; stock 5 → 3; 1 line. Baseline: 500	PASS	EV-G3a
TC-10	Two accepted submissions	Two orders, each deducts once	Distinct ids; 10 → 6; 2 lines	PASS	EV-G3a
TC-11	Raw negative stock write	Rejected by CK_Products_Stock_NonNegative	SqlException naming constraint; stock 1	PASS	EV-G3a, EV-A2
TC-12	10 concurrent cancels of one order across 2 instances	1 × 204, 9 × 409; stock restored once	1 / 9; stock 10. Pre-refactor: 6 successes	PASS	EV-G3a
TC-13	Live: 300 orders / 60 concurrent across 2 API processes, stock 80	Accepted units = 80; stock 0	41 via :5080 + 39 via :5081 = 80; stock 0; consistent=true	PASS	EV-B3
TC-14	Live: repo stress script, 40 requests	Units sold = initial stock; stock 0	After: 29 → 0, 29 lines. Baseline: 30 → 13, 40 lines	PASS	EV-B4, EV-B0
TC-15	Client retry after timeout (idempotency key)	No duplicate order	No idempotency key exists — not tested	NOT RUN	Section 10
TC-16	Multi-line orders in opposite product order (deadlock)	Clean 409 / retry	Not executed	NOT RUN	Section 10
Inventory performance					
TC-20	List latency before optimisation	Baseline recorded under fixed conditions	Median 224.9 ms, p95 261.0 ms	PASS	EV-C3
TC-21	List latency after optimisation (same conditions)	Materially faster; same payload	Median 11.9 ms, p95 15.5 ms; payload identical	PASS	EV-C4
TC-22	Generated SQL before and after captured	Actual EF command log	202 commands → 2 commands	PASS	EV-C1, EV-C2
TC-23	Constant SQL command count (automated)	2 commands for pageSize=200	2 (baseline 202)	PASS	EV-G3a
TC-24	Execution plan / index behaviour	Seek/ordered scan on PKs; no sort	Ordered PK scan + TOP + nested-loop seek; 6 + 400 reads	PASS	EV-C5, EV-C6
TC-25	Filtering, sorting, pagination correctness	Matches reference query; clamp 200; empty past end	3 tests passed	PASS	EV-G3a
SignalR					
TC-30	Stock update in originating browser	A shows new stock	A: 17 → 16	PASS	EV-D3
TC-31	Stock update in another browser without refresh	B shows new stock without reload	B: 16, 4 ms after A. Baseline: B stayed 41	PASS	EV-D4, EV-D0

Test ID	Test scenario	Expected result	Actual result	Status	Evidence
TC-32	No update after failed order	No frame; B unchanged	0 new frames; B 14 → 14	PASS	EV-D6, EV-F5
TC-33	Connection and reconnection	Rejoin group, refresh data; restart after close	Unit specs pass; live outage not performed	PARTIAL	EV-G4
TC-34	Restock broadcast to every browser (hub test)	Both clients receive	Received	PASS	EV-G3a
TC-35	Client that left the group receives nothing	No event	No event	PASS	EV-G3a
TC-36	Delivery across multiple API instances	Requires backplane	Not executed (known limitation)	NOT RUN	Section 10
Order placement refactoring					
TC-40	Successful order contract	200 `{ id }`; line persisted	Pass before and after refactor	PASS	EV-B1, EV-E3
TC-41	Validation errors	400 with existing messages	Pass before and after	PASS	EV-B1, EV-E3
TC-42	Not found / insufficient stock	404 / 409 with existing messages	Pass before and after	PASS	EV-B1, EV-E3
TC-43	Cancel / restock contracts	204; 409 second cancel; 400/404	Pass before and after	PASS	EV-B1, EV-E3
TC-44	Recent orders endpoint	Placed order listed with total	Pass before and after	PASS	EV-B1, EV-E3
TC-45	Transactional consistency	Failed orders leave no records	See TC-07, TC-08, TC-09	PASS	EV-G3a
Low-stock alerts					
TC-50	Stock remains above threshold	No alert	No alert (20 → 18)	PASS	EV-G3a
TC-51	Stock reaches threshold exactly	No alert	Automated: none. Live: 0 toasts in A and B at 15	PASS	EV-F1
TC-52	Stock falls below threshold	One alert with current values	Automated: 1 alert (13). Live: toast "14 left · reorder at 15"	PASS	EV-F2
TC-53	Alert delivered to all connected clients	Both browsers receive	Hub test 2/2 clients; live toast in A and B; frame captured	PASS	EV-F3, EV-F5
TC-54	Angular notification rendering	Product, stock, threshold shown; dismissable	Karma specs pass; live screenshot	PASS	EV-F2, EV-G4
TC-55	Failed order	No alert	None (automated + live)	PASS	EV-D6
TC-56	Rolled-back multi-line order	No alert for earlier line	None	PASS	EV-G3a
TC-57	10 concurrent orders crossing threshold	Exactly one alert	One alert at 14; baseline: lost updates	PASS	EV-G3a

Test ID	Test scenario	Expected result	Actual result	Status	Evidence
TC-58	Component re-initialisation / navigation	No duplicate notifications or listeners	Karma: 1 listener, 1 toast; live: 1 toast after navigation	PASS	EV-G4, EV-F6
TC-59	Restock above threshold re-arms	Second crossing alerts again	Alerts at 14 then 12	PASS	EV-G3a
Final validation					
TC-70	Backend build	0 errors	Build succeeded, 0 warnings, 0 errors	PASS	EV-G1
TC-71	Frontend production build	Exit 0, within budgets	Exit 0; initial 388.10 kB	PASS	EV-G2
TC-72	Backend automated suite	All pass	42/42	PASS	EV-G3
TC-73	Frontend automated suite	All pass	17/17	PASS	EV-G4
TC-74	Human-operated two-browser test	Manual confirmation	Automated headless two-context test performed instead	NOT RUN	—
TC-75	Staging / production-like deployment	Deploy and smoke test	Not performed	NOT RUN	—
TC-76	Load test on Release build	Capacity baseline	Only Development-mode measurements	NOT RUN	EV-G7

Totals: PASS 45 · NOT RUN 6 · PARTIAL 1 (52 test cases). TC IDs map to named automated tests listed in `EV-G3a_backend_test_case_results.txt` and the Karma log.

4.2 Test Result Classification

Status	Meaning
PASS	Executed; observed result matched the expected result.
FAIL	Executed; observed result did not match.
BLOCKED	Could not be completed because of a documented dependency or environment issue.
NOT RUN	Not executed.
PARTIAL	Only part of the required verification was completed.

No result in this report is based on code inspection alone. No test is currently FAIL or BLOCKED. Failures shown for the baseline (commit fc3c873) are the reproductions of the reported defects, not failures of this release.

5. Screenshots and Test Evidence

Screenshots were captured from the running application in two isolated headless Chrome browser contexts (separate storage and SignalR connections, viewport 1440 × 900) on 29 Oct 2026 03:18 UTC, using the Chrome DevTools Protocol. They are unedited. Log excerpts are verbatim extracts from the archived files named in each block (long lines truncated with "..."). All artifacts are in `release/Release_Evidence/`; the index is `Evidence_Index.md`.

5.1 Evidence A — Application Startup

EV-A1 Docker Compose first start: images, health, migrations, seed

```
 Image mcr.microsoft.com/mssql/server:2022-latest Pulled
 Image stage-3-assessment-api Built
 Image stage-3-assessment-web Built
 Container stage-3-assessment-api-1 Created
 Container stage-3-assessment-web-1 Created
 Container stage-3-assessment-db-1 Healthy
 Container stage-3-assessment-api-1 Started
 Container stage-3-assessment-web-1 Started
web-1  | Application bundle generation complete. [2.623 seconds] - 2026-10-29T02:34:43.360Z
web-1  |   ➜  Local:   http://localhost:4200/
api-1  |       Applying migration '20261023081746_InitialCreate'.
api-1  |       WITH numbers AS (
api-1  |       Now listening on: http://0.0.0.0:8080
api-1  |       Application started. Press Ctrl+C to shut down.
api-1  |       Hosting environment: Development
api-1  |       Applying migration '20261029024124_AddNonNegativeStockConstraint'.
api-1  |       Now listening on: http://0.0.0.0:8080
```

Log excerpt EV-A1 — Docker Compose first start: images, health, migrations, seed

Evidence ID	**EV-A1**
Related issue / test	Installation; all issues
Evidence type	Log excerpt (docker compose up --build)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 02:30–02:52 UTC
Artifact	`logs/EV-A1_docker_compose_first_start_excerpt.log`
What it demonstrates	Documented startup command builds/pulls images; db reports Healthy before api starts; EF applies migrations; seeding runs the 250k-order insert; API and Angular dev server start.
Observation	All three containers started; InitialCreate and later AddNonNegativeStockConstraint applied (after API restart); web bundle generated at http://localhost:4200.
Result / status	**PASS** Startup workflow from README verified.

EV-A2 Service health, seeded data and integrity state

```
# Captured 2026-10-29T03:19:20Z on the local Docker Compose stack
$ docker compose ps
SERVICE   STATUS                    PORTS
api       Up 28 minutes             0.0.0.0:5080->8080/tcp, [::]:5080->8080/tcp
db        Up 41 minutes (healthy)   0.0.0.0:1433->1433/tcp, [::]:1433->1433/tcp, 0.0.0.0:14333->1433/tcp, [::]:1433...
web       Up 44 minutes             0.0.0.0:4200->4200/tcp, [::]:4200->4200/tcp

$ curl http://localhost:5080/health
{"status":"healthy"}
$ curl -o /dev/null -w '%{http_code}' http://localhost:4200/
200

-- seeded data (SQL Server)
Products 2000
Categories 20
Orders 251803
Applied migration 20261023081746_InitialCreate
Applied migration 20261029024124_AddNonNegativeStockConstraint
Check constraint CK_Products_Stock_NonNegative ([Stock]>=(0))
Products with negative stock 0
Orders without lines (placed after seed) 0
```

Log excerpt EV-A2 — Service health, seeded data and integrity state

Evidence ID	**EV-A2**
Related issue / test	Installation; INV-001
Evidence type	Command output + SQL results
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:19 UTC
Artifact	`logs/EV-A2_services_health_and_seed_state.txt`
What it demonstrates	Containers running, API health endpoint, web app reachable, seed counts, applied migrations, CHECK constraint and integrity counters after all testing.
Observation	/health returned {"status":"healthy"}; web 200; 2,000 products, 20 categories; both migrations applied; 0 products with negative stock; 0 orders without lines.
Result / status	**PASS**

5.2 Evidence B — Inventory Concurrency (INV-001)

EV-B1 Baseline reproduction on the original code

```
[xUnit.net 00:00:06.59]         SQL commands executed: 202
Failed Inventory.Api.Tests.ProductListTests.Inventory_page_request_uses_a_constant_number_of_sql_commands [279 ms]
SQL commands executed: 202
[xUnit.net 00:00:10.76]         Last-unit race results: OK, OK
[xUnit.net 00:00:11.48]         40 requests: 40 OK, 0 Conflict, other 0; final stock 5; units in order lines 40
Failed Inventory.Api.Tests.OrderConcurrencyTests.Two_instances_racing_for_the_last_unit_cannot_both_succeed [238 ms]
Last-unit race results: OK, OK
Failed Inventory.Api.Tests.OrderConcurrencyTests.Concurrent_orders_of_mixed_quantities_keep_stock_and_order_lines...
Failed Inventory.Api.Tests.OrderConcurrencyTests.Unknown_product_on_a_later_line_rolls_back_earlier_deductions [1...
Failed Inventory.Api.Tests.OrderConcurrencyTests.Many_concurrent_orders_across_two_instances_never_exceed_stock [...
40 requests: 40 OK, 0 Conflict, other 0; final stock 5; units in order lines 40
Failed Inventory.Api.Tests.OrderConcurrencyTests.Failure_on_a_later_line_rolls_back_earlier_deductions [32 ms]
Failed Inventory.Api.Tests.OrderConcurrencyTests.Notification_failure_after_commit_does_not_report_a_committed_or...
Failed Inventory.Api.Tests.LowStockAlertTests.Concurrent_orders_crossing_the_threshold_raise_exactly_one_alert [1...
Failed Inventory.Api.Tests.StockHubTests.Order_placed_by_one_browser_updates_stock_in_the_other_browser [5 s]
Total tests: 40
Passed: 31
Failed: 9
```

Log excerpt EV-B1 — Baseline reproduction on the original code

Evidence ID	**EV-B1**
Related issue / test	INV-001, INV-002, INV-003, INV-005 · TC-04, TC-05, TC-23, TC-31
Evidence type	Automated test log (xUnit, detailed)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:21 UTC
Artifact	`logs/EV-B1_backend_tests_BASELINE_before_fixes.log`
What it demonstrates	The same test suite run against the original application code (worktree of 6028cbd: original code + tests) to reproduce the defects before any fix.
Observation	40 tests: 31 passed, 9 failed. Last-unit race: both instances returned OK. 40 orders accepted for stock 10 (final stock 5). Inventory page: 202 SQL commands. Cross-browser stock update not delivered.
Result / status	**PASS** Defects reproduced (expected failures on baseline).

EV-B2 Fixed code: last-unit race and concurrent burst across two API instances

```
Last-unit race results: OK, Conflict
40 requests: 10 OK, 30 Conflict, other 0; final stock 0; units in order lines 10


Passed OrderConcurrencyTests.Concurrent_cancellations_return_stock_only_once [226 ms]
Passed OrderConcurrencyTests.Concurrent_orders_of_mixed_quantities_keep_stock_and_order_lines_c...
Passed OrderConcurrencyTests.Database_rejects_negative_stock_even_from_a_raw_update [30 ms]
Passed OrderConcurrencyTests.Each_accepted_submission_deducts_stock_exactly_once [201 ms]
Passed OrderConcurrencyTests.Failure_on_a_later_line_rolls_back_earlier_deductions [58 ms]
Passed OrderConcurrencyTests.Many_concurrent_orders_across_two_instances_never_exceed_stock [39...
Passed OrderConcurrencyTests.Notification_failure_after_commit_does_not_report_a_committed_orde...
Passed OrderConcurrencyTests.Ordering_exactly_the_available_stock_succeeds_and_leaves_zero [45 ms]
Passed OrderConcurrencyTests.Ordering_less_than_available_stock_deducts_exactly_the_ordered_qua...
Passed OrderConcurrencyTests.Ordering_more_than_available_stock_is_rejected_without_side_effect...
Passed OrderConcurrencyTests.Two_instances_racing_for_the_last_unit_cannot_both_succeed [287 ms]
Passed OrderConcurrencyTests.Unknown_product_on_a_later_line_rolls_back_earlier_deductions [44 ms]
```

Log excerpt EV-B2 — Fixed code: last-unit race and concurrent burst across two API instances

Evidence ID	**EV-B2**
Related issue / test	INV-001 · TC-01…TC-12
Evidence type	Automated test log (xUnit, detailed)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`logs/EV-G3_backend_tests_FINAL_42_passed.log; logs/EV-G3a_backend_test_case_results.txt`
What it demonstrates	Two independent API hosts on one SQL Server database. The last-unit test holds a row lock until SQL Server reports both requests blocked, then releases it so both compete at the same instant.
Observation	Last-unit race: OK, Conflict; 40 requests on stock 10: 10 OK, 30 Conflict, final stock 0, 10 units in order lines. All 12 concurrency tests passed.
Result / status	**PASS**

EV-B3 Live: two API processes sharing the database

```
mode=orders requests=200 concurrency=50 instances=http://localhost:5080,http://localhost:5081
  200 via 5080: 26
  409 via 5080: 74
  409 via 5081: 100
  throughput=259.3 req/s  p50=51.6ms p95=544.0ms p99=598.8ms max=600.0ms
  stock before=26 after=0 accepted units=26 consistent=true
mode=orders requests=300 concurrency=60 instances=http://localhost:5080,http://localhost:5081
  200 via 5080: 41
  200 via 5081: 39
  409 via 5080: 109
  409 via 5081: 111
  throughput=180.6 req/s  p50=118.8ms p95=1072.2ms p99=1190.8ms max=1218.2ms
  stock before=80 after=0 accepted units=80 consistent=true
```

Log excerpt EV-B3 — Live: two API processes sharing the database

Evidence ID	**EV-B3**
Related issue / test	INV-001 · TC-13
Evidence type	Load generator output + SQL
Environment	Docker API on :5080 + separate host API process on :5081, same SQL Server database
Captured	29 Oct 2026
Artifact	`logs/EV-B3_two_api_instances_load_test.txt`
What it demonstrates	Real separate processes; orders distributed round-robin. First run had a cold second instance, so it was repeated warm.
Observation	Warm run: 300 requests, 60 concurrent, stock 80 → 41 accepted via :5080 and 39 via :5081 (80 total), 220 × 409, stock 0, consistent=true.
Result / status	**PASS**

EV-B4 Repository stress script before and after the fix

```
# Captured 2026-10-29T03:22:11Z — seed/stress-orders.sh against the ORIGINAL code (commit fc3c873 code via worktree...
-- before
stock 30
order lines 0 0
$ API_URL=http://localhost:5082 PRODUCT_ID=10 REQUESTS=40 ./seed/stress-orders.sh
Concurrent requests: 40
      3 200
-- after
stock 13
order lines 40 40



# Captured 2026-10-29T03:20:41Z — repository script seed/stress-orders.sh against the fixed API (localhost:5080)
-- before
stock 29
order lines 0 0
$ PRODUCT_ID=9 REQUESTS=40 ./seed/stress-orders.sh
Concurrent requests: 40
      5 200
      3 409
-- after
stock 0
order lines 29 29
negative stock rows 0
```

Log excerpt EV-B4 — Repository stress script before and after the fix

Evidence ID	**EV-B4**
Related issue / test	INV-001 · TC-14
Evidence type	Script output + SQL
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:20–03:22 UTC
Artifact	`logs/EV-B0_stress_orders_script_BASELINE.txt; logs/EV-B4_stress_orders_script_after_fix.txt`
What it demonstrates	`seed/stress-orders.sh` (40 concurrent single-unit orders). Baseline run against the original code on :5082; fixed run against :5080. Database counts are authoritative (the script's own tally under-counts on Git Bash for Windows).
Observation	Baseline: stock 30 → 13 while 40 units were recorded in order lines (23 units oversold). Fixed: stock 29 → 0 with exactly 29 units recorded; 0 rows with negative stock.
Result / status	**PASS**

5.3 Evidence C — Inventory Performance (INV-002)

EV-C1 Generated SQL before the fix (202 commands)

```
Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT COUNT(*)
FROM [Products] AS [p]
Executed DbCommand (1ms) [Parameters=[@__p_0='?' (DbType = Int32), @__p_1='?' (DbType = Int32)], CommandType=...
SELECT [p].[Id], [p].[CategoryId], [p].[Name], [p].[ReorderThreshold], [p].[RowVersion], [p].[Sku], [p].[Stoc...
FROM [Products] AS [p]
ORDER BY [p].[Id]
OFFSET @__p_0 ROWS FETCH NEXT @__p_1 ROWS ONLY
Executed DbCommand (1ms) [Parameters=[@__product_CategoryId_0='?' (DbType = Int32)], CommandType='Text', Comm...
SELECT TOP(2) [c].[Name]
FROM [Categories] AS [c]
WHERE [c].[Id] = @__product_CategoryId_0
Executed DbCommand (1ms) [Parameters=[@__product_CategoryId_0='?' (DbType = Int32)], CommandType='Text', Comm...
SELECT TOP(2) [c].[Name]
FROM [Categories] AS [c]
WHERE [c].[Id] = @__product_CategoryId_0
Executed DbCommand (1ms) [Parameters=[@__product_CategoryId_0='?' (DbType = Int32)], CommandType='Text', Comm...
```

Log excerpt EV-C1 — Generated SQL before the fix (202 commands)

Evidence ID	**EV-C1**
Related issue / test	INV-002 · TC-22
Evidence type	EF Core command log (docker compose logs api)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`logs/EV-C1_inventory_list_SQL_BEFORE_202_commands.log`
What it demonstrates	All SQL executed for one GET /api/products?pageSize=200 on the seeded database.
Observation	COUNT, a page query selecting all entity columns including RowVersion, then one Categories lookup per row — 202 Executed DbCommand entries in total.
Result / status	**PASS** Root cause (N+1) confirmed.

EV-C2 Generated SQL after the fix (2 commands)

```
Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
SELECT COUNT(*)
FROM [Products] AS [p]
Executed DbCommand (3ms) [Parameters=[@__p_0='?' (DbType = Int32), @__p_1='?' (DbType = Int32)], CommandType=...
SELECT [p0].[Id], [p0].[Name], [p0].[Sku], [c].[Name], [p0].[UnitPrice], [p0].[Stock], [p0].[ReorderThreshold]
FROM (
SELECT [p].[Id], [p].[CategoryId], [p].[Name], [p].[ReorderThreshold], [p].[Sku], [p].[Stock], [p].[UnitP...
FROM [Products] AS [p]
ORDER BY [p].[Id]
OFFSET @__p_0 ROWS FETCH NEXT @__p_1 ROWS ONLY
) AS [p0]
INNER JOIN [Categories] AS [c] ON [p0].[CategoryId] = [c].[Id]
ORDER BY [p0].[Id]
```

Log excerpt EV-C2 — Generated SQL after the fix (2 commands)

Evidence ID	**EV-C2**
Related issue / test	INV-002 · TC-22, TC-23
Evidence type	EF Core command log
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`logs/EV-C2_inventory_list_SQL_AFTER_2_commands.log`
What it demonstrates	Same request after the projection change.
Observation	2 commands: COUNT and one page query joining Categories and selecting only DTO columns.
Result / status	**PASS**

EV-C3 Latency before and after; statistics and execution plan

```
BEFORE (baseline ProductsController.List)
page (pageSize=200): n=50 min=166.2ms median=224.9ms p95=261.0ms mean=222.6ms
search=Milk:         n=50 min=69.8ms median=95.4ms p95=114.7ms mean=96.1ms
pageSize=50 page=3: n=50 min=54.6ms median=68.6ms p95=88.7ms mean=71.0ms
payload bytes (pageSize=200): 28978


AFTER (projected single query)
page (pageSize=200): n=50 min=8.9ms median=11.9ms p95=15.5ms mean=12.2ms
search=Milk:         n=50 min=19.7ms median=22.8ms p95=29.9ms mean=23.9ms
pageSize=50 page=3: n=50 min=5.7ms median=7.2ms p95=8.7ms mean=7.5ms
payload bytes (pageSize=200): 28978



== optimized page query
Table 'Categories'. Scan count 0, logical reads 400
Table 'Products'. Scan count 1, logical reads 6
CPU time = 4 ms,  elapsed time = 3 ms.
== search count (LIKE %Milk%)
Table 'Products'. Scan count 1, logical reads 41
CPU time = 8 ms,  elapsed time = 9 ms.
== table sizes


  |--Nested Loops(Inner Join, OUTER REFERENCES:([p].[CategoryId]))
       |--Top(TOP EXPRESSION:((200)))
       |    |--Clustered Index Scan(OBJECT:([Products].[PK_Products] AS [p]), ORDERED FORWARD)
       |--Clustered Index Seek(OBJECT:([Categories].[PK_Categories] AS [c]), SEEK:([c].[Id]=[Inve...
```

Log excerpt EV-C3 — Latency before and after; statistics and execution plan

Evidence ID	**EV-C3**
Related issue / test	INV-002 · TC-20, TC-21, TC-24
Evidence type	Benchmark output, STATISTICS IO/TIME, SHOWPLAN_TEXT
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`logs/EV-C3…EV-C6 (latency before/after, statistics, plan)`
What it demonstrates	50 sequential requests after 5 warm-ups per URL, API container restarted before each series, same seeded database; SQL Server statistics and text plan for the optimised query.
Observation	Inventory page median 224.9 → 11.9 ms; payload unchanged. Products 6 logical reads; Categories 400 (200 seeks); plan has no sort and no scan of Categories.
Result / status	**PASS**

5.4 Evidence D — SignalR Synchronisation (INV-003)

EV-D0 Baseline: Browser B not updated

```
02:47:03.461 Product FPR-000021 "Hass Avocados 4 Pack" stock 41, threshold 15
02:47:04.273 Browser A: connection live, shows stock 41
02:47:04.380 Browser B: connection live, shows stock 41
02:47:04.388 Browser A: clicked Order (qty 1), expecting stock 40
02:47:04.733 Browser A: shows stock 40
02:47:09.777 Browser B: Browser B: timed out waiting for stock 40 (last value 41)  -> FAIL (not updated)
02:47:09.897 Browser A: failed order shows "Order could not be placed."
02:47:11.445 Browser B after failed order: 41 (before 41, database 40)  -> FAIL
```

Log excerpt EV-D0 — Baseline: Browser B not updated

Evidence ID	**EV-D0**
Related issue / test	INV-003 · TC-31 (baseline)
Evidence type	Headless two-browser check output
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 02:47 UTC
Artifact	`logs/EV-D0_two_browser_check_BASELINE_fail.txt`
What it demonstrates	seed/two-browser-check.mjs against the original notifier code.
Observation	Browser A showed 40; Browser B stayed at 41 (database 40).
Result / status	**PASS** Defect reproduced.

EV-D1 Browser A — initial stock

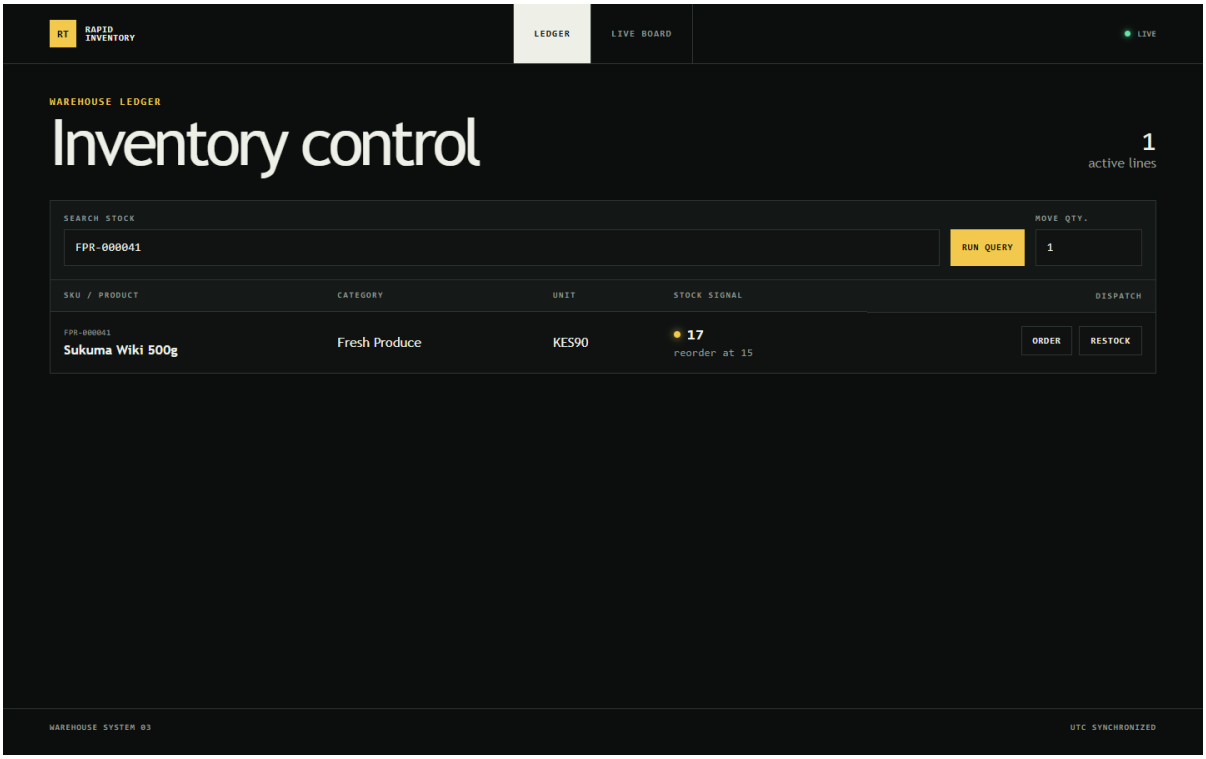


Figure EV-D1 — Browser A — initial stock

Evidence ID	**EV-D1**
Related issue / test	INV-003 · TC-30
Evidence type	Screenshot (headless Chrome, context A)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:33 UTC
Artifact	`screenshots/EV-D1_browserA_initial_stock.png`
What it demonstrates	Inventory page filtered to FPR-000041 "Sukuma Wiki 500g" with live SignalR connection (status LIVE).
Observation	Stock 17, reorder at 15.
Result / status	**PASS**

EV-D2 Browser B — same initial stock

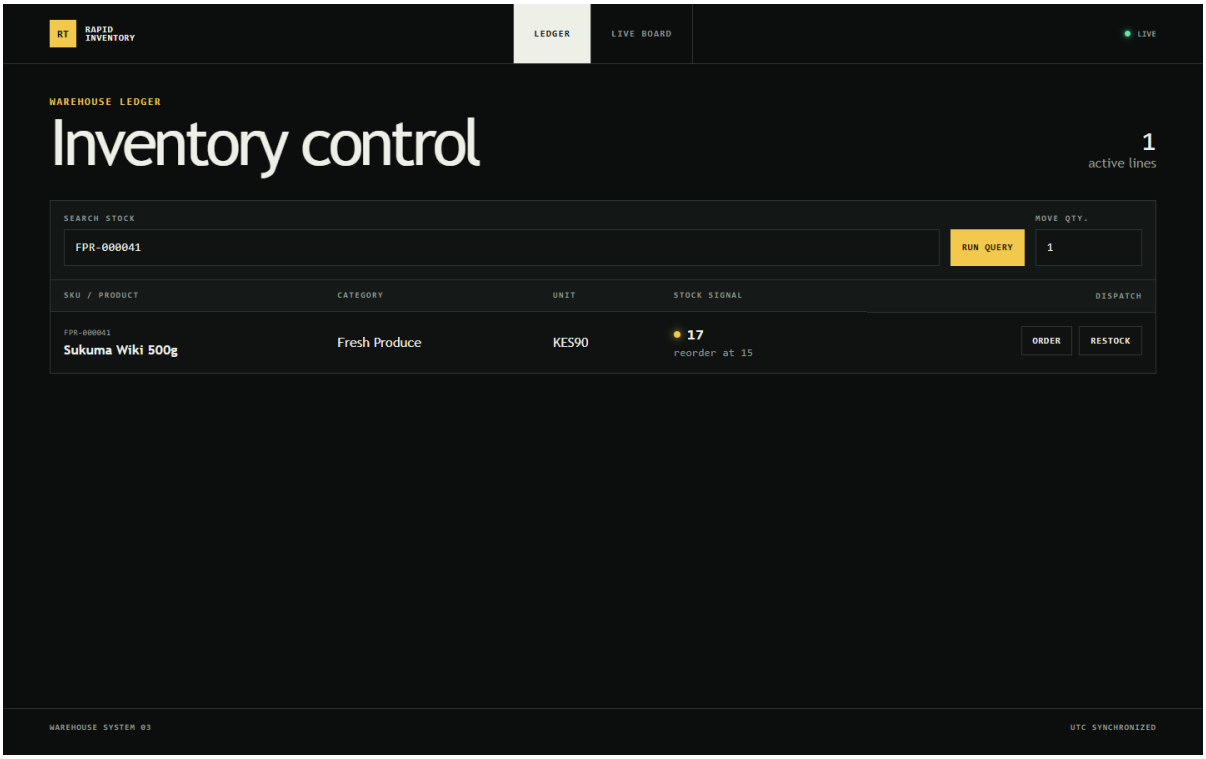


Figure EV-D2 — Browser B — same initial stock

Evidence ID	**EV-D2**
Related issue / test	INV-003 · TC-31
Evidence type	Screenshot (headless Chrome, context B)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:33 UTC
Artifact	`screenshots/EV-D2_browserB_initial_stock.png`
What it demonstrates	Second, isolated browser context with its own SignalR connection.
Observation	Stock 17 — identical to Browser A.
Result / status	**PASS**

EV-D3 Browser A — after placing an order

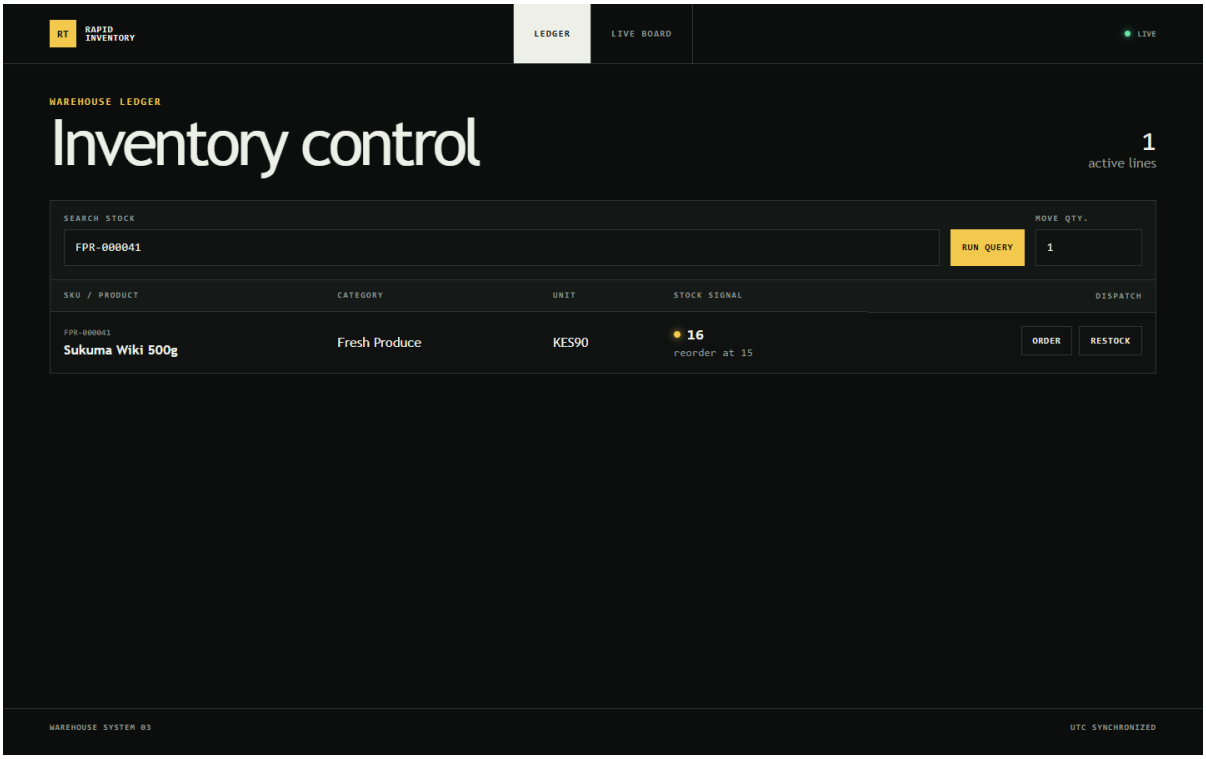


Figure EV-D3 — Browser A — after placing an order

Evidence ID	**EV-D3**
Related issue / test	INV-003 · TC-30
Evidence type	Screenshot
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:33 UTC
Artifact	`screenshots/EV-D3_browserA_after_order.png`
What it demonstrates	Order (qty 1) placed by clicking Order in Browser A.
Observation	Stock 16.
Result / status	**PASS**

EV-D4 Browser B — updated without refresh

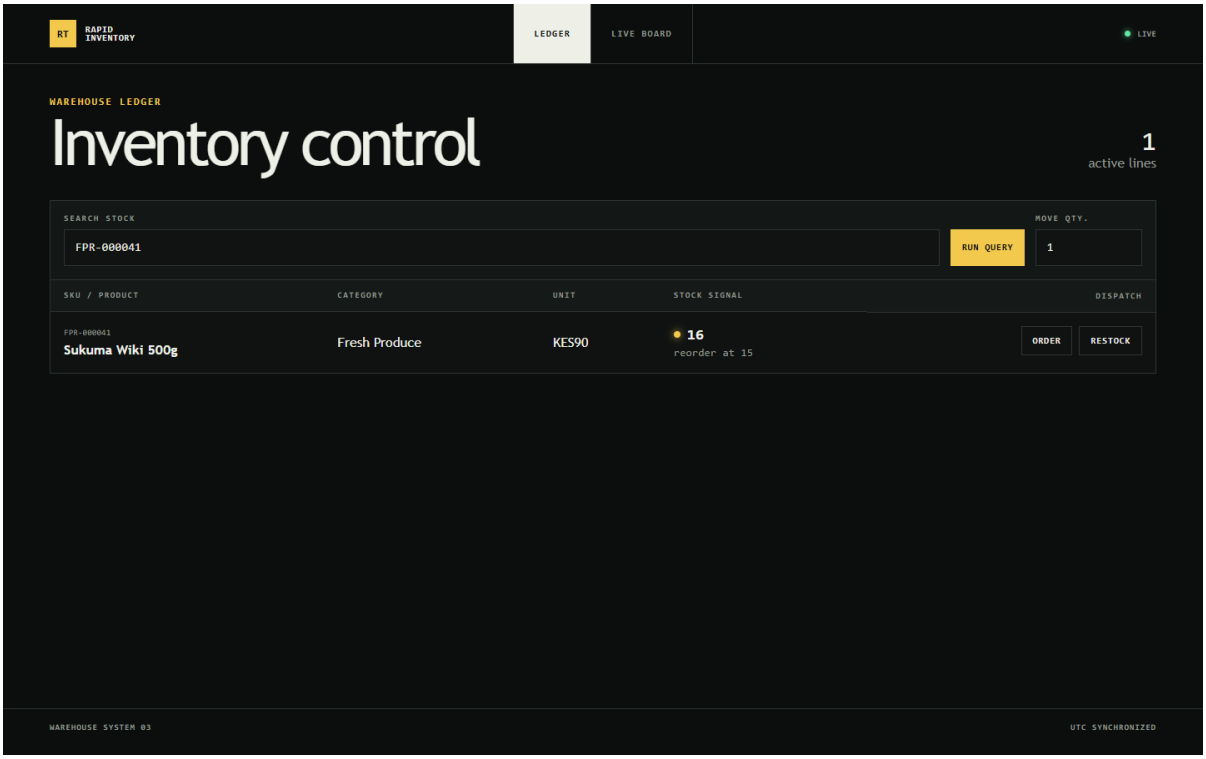


Figure EV-D4 — Browser B — updated without refresh

Evidence ID	**EV-D4**
Related issue / test	INV-003 · TC-31
Evidence type	Screenshot
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:33 UTC
Artifact	`screenshots/EV-D4_browserB_updated_without_refresh.png`
What it demonstrates	Browser B was not reloaded; the value arrives via the StockUpdated SignalR event.
Observation	Stock 16, displayed 4 ms after Browser A (capture-log.txt).
Result / status	**PASS**

EV-D5 Browser A — failed order (quantity 100000)

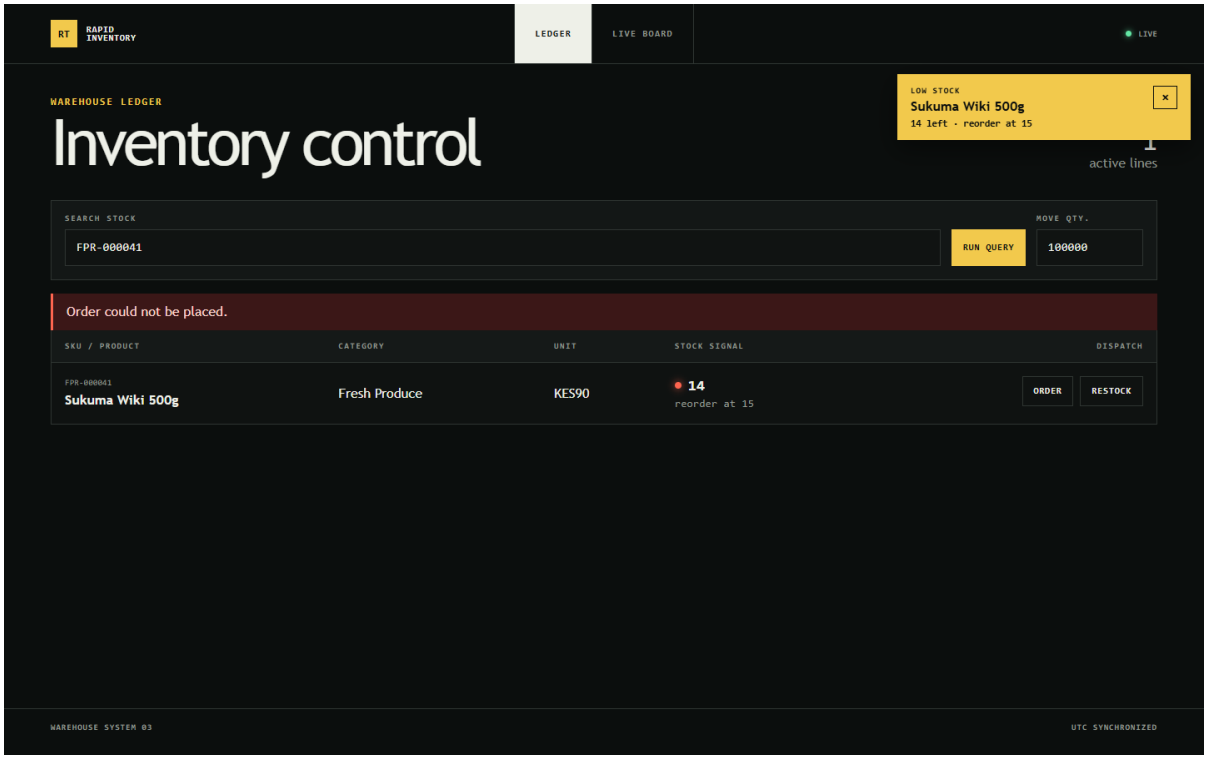


Figure EV-D5 — Browser A — failed order (quantity 100000)

Evidence ID	**EV-D5**
Related issue / test	INV-003 · TC-32; INV-005 · TC-55
Evidence type	Screenshot
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:37 UTC
Artifact	`screenshots/EV-D5_browserA_failed_order_error.png`
What it demonstrates	An order larger than the stock is rejected by the API (409).
Observation	Error banner "Order could not be placed."; stock remains 14.
Result / status	**PASS**

EV-D6 Browser B — unchanged after the failed order

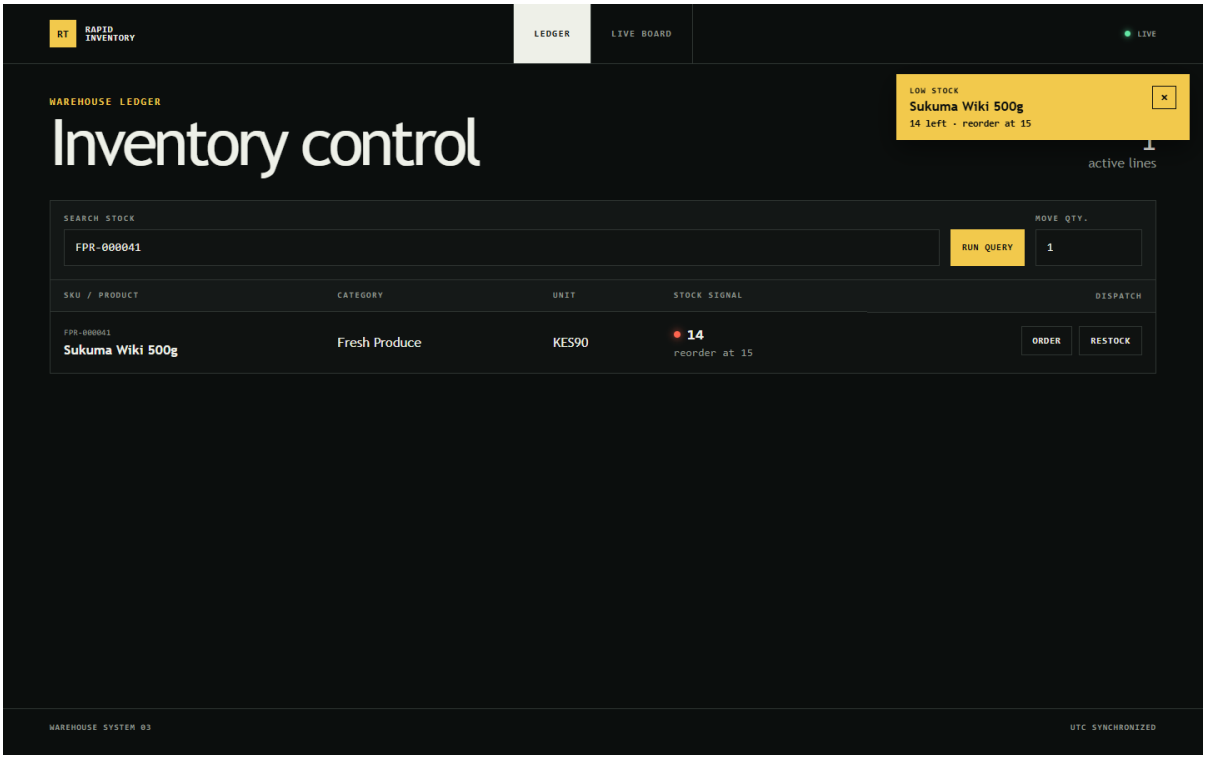


Figure EV-D6 — Browser B — unchanged after the failed order

Evidence ID	**EV-D6**
Related issue / test	INV-003 · TC-32; INV-005 · TC-55
Evidence type	Screenshot + frame count
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:37 UTC
Artifact	`screenshots/EV-D6_browserB_unchanged_after_failed_order.png; screenshots/capture-log.txt`
What it demonstrates	No stock update or alert should be published for a rejected order.
Observation	Stock 14 → 14; still one toast (from the earlier valid crossing); 0 new SignalR frames received by Browser B.
Result / status	**PASS**

5.5 Evidence E — Order Placement Refactoring (INV-004)

EV-E1 Original controller (fc3c873) — business logic in Place()

```
[HttpPost]
public async Task<IActionResult> Place(PlaceOrderRequest request, CancellationToken ct)
{
    if (request.Lines.Count == 0)
        return BadRequest(new { error = "Order must contain at least one line." });

    if (request.Lines.Any(line => line.Quantity <= 0))
        return BadRequest(new { error = "All line quantities must be greater than zero." });

    var order = new Order { OrderDate = DateTime.UtcNow, Status = OrderStatus.Placed };
    var stockChanges = new List<(int Id, string Name, int Previous, int Current, int Threshold)>();

    foreach (var line in request.Lines)
    {
        var product = await db.Products
            .Where(p => p.Id == line.ProductId)
            .Select(p => new { p.Id, p.Name, p.UnitPrice, p.Stock, p.ReorderThreshold })
            .SingleOrDefaultAsync(ct);

        if (product is null)
            return NotFound(new { error = $"Product {line.ProductId} not found." });

        if (product.Stock < line.Quantity)
            return Conflict(new { error = $"Insufficient stock for product {line.ProductId}." });

        var newStock = product.Stock - line.Quantity;
        await db.Products
            .Where(p => p.Id == line.ProductId)
            .ExecuteUpdateAsync(update => update.SetProperty(p => p.Stock, newStock), ct);
```

Source excerpt EV-E1 — Original controller (fc3c873) — business logic in Place()

Evidence ID	**EV-E1**
Related issue / test	INV-004
Evidence type	Source excerpt
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`code/EV-E1_OrdersController_BEFORE_fc3c873.cs`
What it demonstrates	Validation, stock read/check, absolute update, order construction, persistence and SignalR publishing all inside the action.
Observation	≈ 50 lines of business logic in Place(); similar logic in Cancel() and Restock().
Result / status	**PASS** Design problem confirmed.

EV-E2 Refactored controller — HTTP concerns only

```
public class OrdersController(AppDbContext db, OrderService orders) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> Place(PlaceOrderRequest request, CancellationToken ct)
        => ToActionResult(await orders.PlaceOrderAsync(request, ct));


    [HttpPost("{id:int}/cancel")]
    public async Task<IActionResult> Cancel(int id, CancellationToken ct)
        => ToActionResult(await orders.CancelOrderAsync(id, ct));


    [HttpPost("/api/products/{productId:int}/restock")]
    public async Task<IActionResult> Restock(int productId, RestockRequest request, CancellationToken ct)
        => ToActionResult(await orders.RestockAsync(productId, request, ct));
    ...


    private IActionResult ToActionResult(ServiceResult result) => result switch
    {
        ServiceResult.Ok<int> created => Ok(new { id = created.Value }),
        ServiceResult.Ok => NoContent(),
        ServiceResult.Invalid invalid => BadRequest(new { error = invalid.Error }),
        ServiceResult.NotFound notFound => NotFound(new { error = notFound.Error }),
        ServiceResult.Conflict conflict => Conflict(new { error = conflict.Error }),
        _ => throw new InvalidOperationException($"Unhandled service result {result.GetType().Name}.")
    };
```

Source excerpt EV-E2 — Refactored controller — HTTP concerns only

Evidence ID	**EV-E2**
Related issue / test	INV-004 · TC-40…TC-44
Evidence type	Source excerpt
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`code/EV-E2_OrdersController_AFTER.cs`
What it demonstrates	Actions delegate to OrderService and map ServiceResult to the same HTTP responses as before.
Observation	Place/Cancel/Restock are one-line delegations; Recent (read-only query) unchanged.
Result / status	**PASS**

EV-E3 Application service and regression tests

```
    public async Task<ServiceResult> PlaceOrderAsync(PlaceOrderRequest request, CancellationToken ct = default)
    {
        var validation = ValidateOrder(request);
        if (validation is not null) return validation;

        var order = new Order { OrderDate = DateTime.UtcNow, Status = OrderStatus.Placed };
        var changes = new List<StockChange>();

        // Deductions and the order insert commit or roll back together. Disposing the transaction
        // without committing (any early return or exception) rolls back every deduction made so far.
        await using var transaction = await db.Database.BeginTransactionAsync(ct);

        foreach (var line in request.Lines)
        {
            // Check-and-deduct is a single conditional UPDATE. SQL Server takes the row's update lock
            // before evaluating "Stock >= @quantity", so concurrent requests on any API instance are
            // serialized on the row and a request can only deduct stock that is still there.
            var deducted = await db.Products
                .Where(p => p.Id == line.ProductId && p.Stock >= line.Quantity)
                .ExecuteUpdateAsync(s => s.SetProperty(p => p.Stock, p => p.Stock - line.Quantity), ct);

            if (deducted == 0)

Passed OrderContractTests.Cancel_of_unknown_order_returns_404 [22 ms]
Passed OrderContractTests.Cancel_returns_stock_and_a_second_cancel_returns_409 [77 ms]
Passed OrderContractTests.Order_exceeding_stock_returns_409 [45 ms]
Passed OrderContractTests.Order_for_unknown_product_returns_404 [14 ms]
Passed OrderContractTests.Order_with_non_positive_quantity_returns_400(quantity: -3) [16 ms]
Passed OrderContractTests.Order_with_non_positive_quantity_returns_400(quantity: 0) [42 ms]
Passed OrderContractTests.Order_with_two_lines_for_the_same_product_deducts_both [63 ms]
Passed OrderContractTests.Order_without_lines_returns_400 [4 ms]
Passed OrderContractTests.Recent_orders_returns_placed_orders_with_lines_and_totals [190 ms]
Passed OrderContractTests.Restock_adds_quantity_and_returns_204 [34 ms]
Passed OrderContractTests.Restock_of_unknown_product_returns_404 [13 ms]
Passed OrderContractTests.Restock_with_non_positive_quantity_returns_400 [18 ms]
Passed OrderContractTests.Successful_order_returns_200_with_the_new_order_id [159 ms]
```

Source excerpt EV-E3 — Application service and regression tests

Evidence ID	**EV-E3**
Related issue / test	INV-004 · TC-40…TC-45
Evidence type	Source excerpt + test results
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`code/EV-E4_OrderService_AFTER.cs; logs/EV-G3a_backend_test_case_results.txt`
What it demonstrates	OrderService owns the transactional use case; 13 contract tests confirm unchanged HTTP behaviour.
Observation	All 13 OrderContractTests passed on the original controller (EV-B1) and after the refactor.
Result / status	**PASS**

5.6 Evidence F — Low-Stock Notifications (INV-005)

EV-F1 Stock exactly at the reorder threshold — no alert

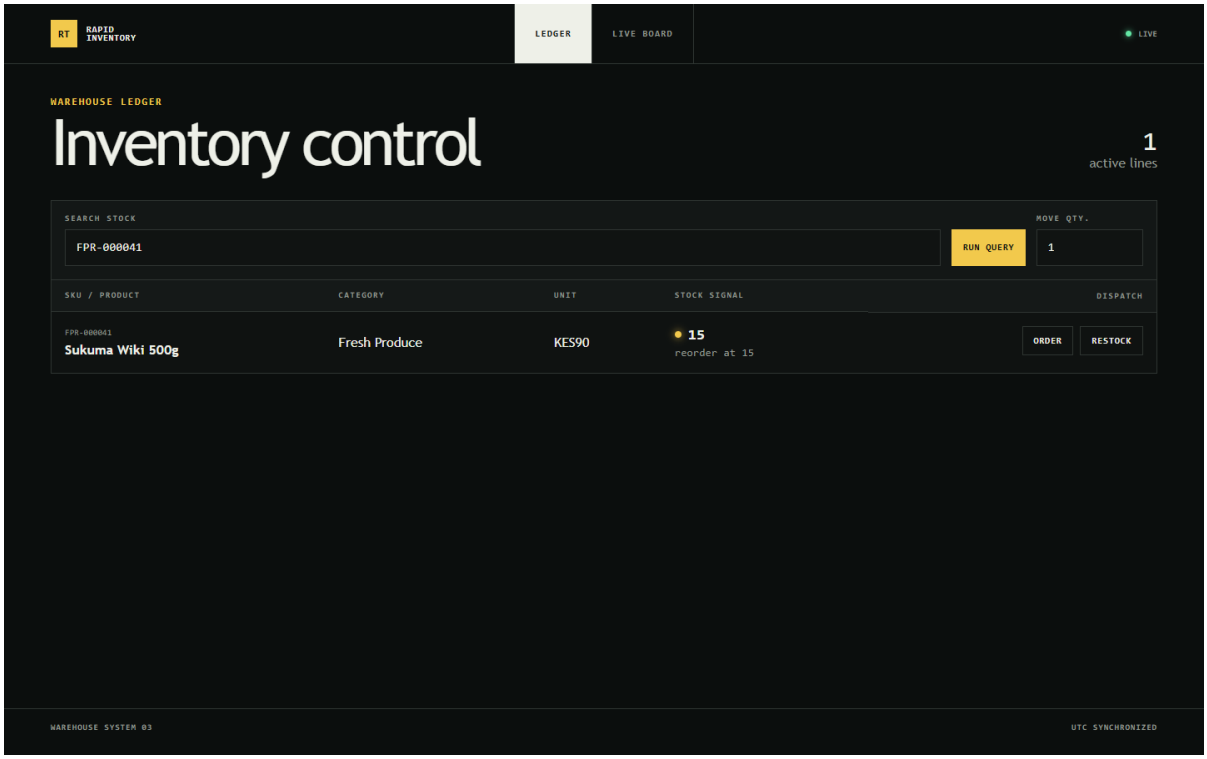


Figure EV-F1 — Stock exactly at the reorder threshold — no alert

Evidence ID	**EV-F1**
Related issue / test	INV-005 · TC-51
Evidence type	Screenshot (Browser B)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:34 UTC
Artifact	`screenshots/EV-F1_browserB_at_threshold_no_alert.png`
What it demonstrates	Product stock 15 with reorder threshold 15 after an order from Browser A.
Observation	No toast in Browser A or B (0 / 0).
Result / status	**PASS**

EV-F2 Stock below threshold — toast in Browser A

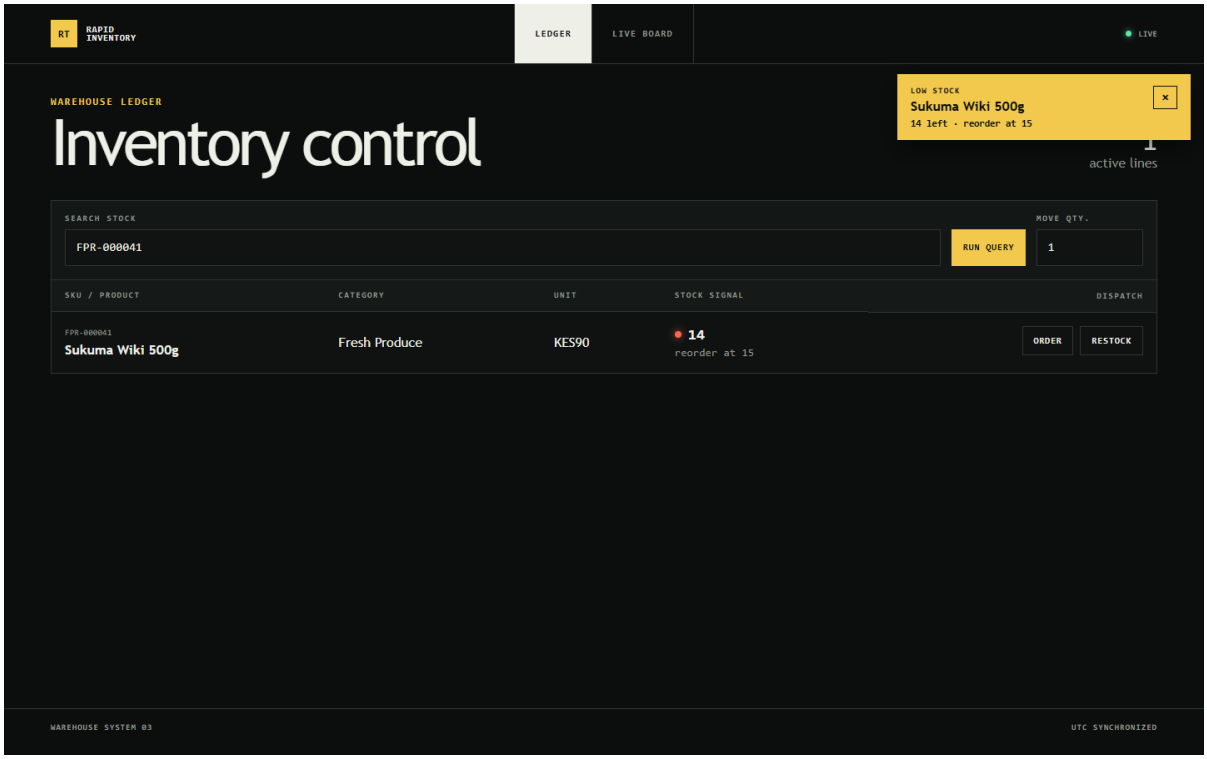


Figure EV-F2 — Stock below threshold — toast in Browser A

Evidence ID	**EV-F2**
Related issue / test	INV-005 · TC-52, TC-54
Evidence type	Screenshot (Browser A)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:35 UTC
Artifact	`screenshots/EV-F2_browserA_low_stock_toast.png`
What it demonstrates	Next order moves stock 15 → 14 (below 15).
Observation	Toast "LOW STOCK · Sukuma Wiki 500g · 14 left · reorder at 15". Cosmetic: the toast overlaps the page-heading line counter.
Result / status	**PASS**

EV-F3 Same alert delivered to Browser B

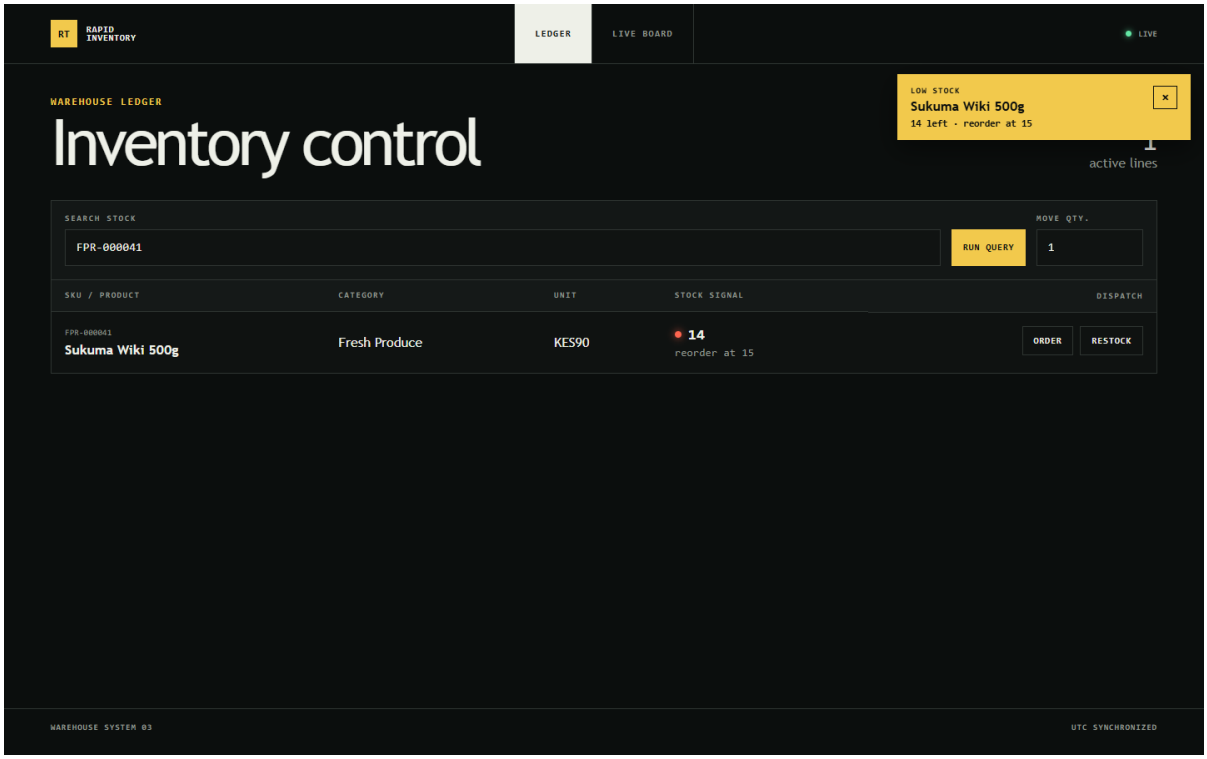


Figure EV-F3 — Same alert delivered to Browser B

Evidence ID	**EV-F3**
Related issue / test	INV-005 · TC-53
Evidence type	Screenshot (Browser B)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:35 UTC
Artifact	`screenshots/EV-F3_browserB_low_stock_toast.png`
What it demonstrates	Browser B did not place the order.
Observation	Identical toast and stock 14 shown.
Result / status	**PASS**

EV-F4 Live board in Browser B lists the alert

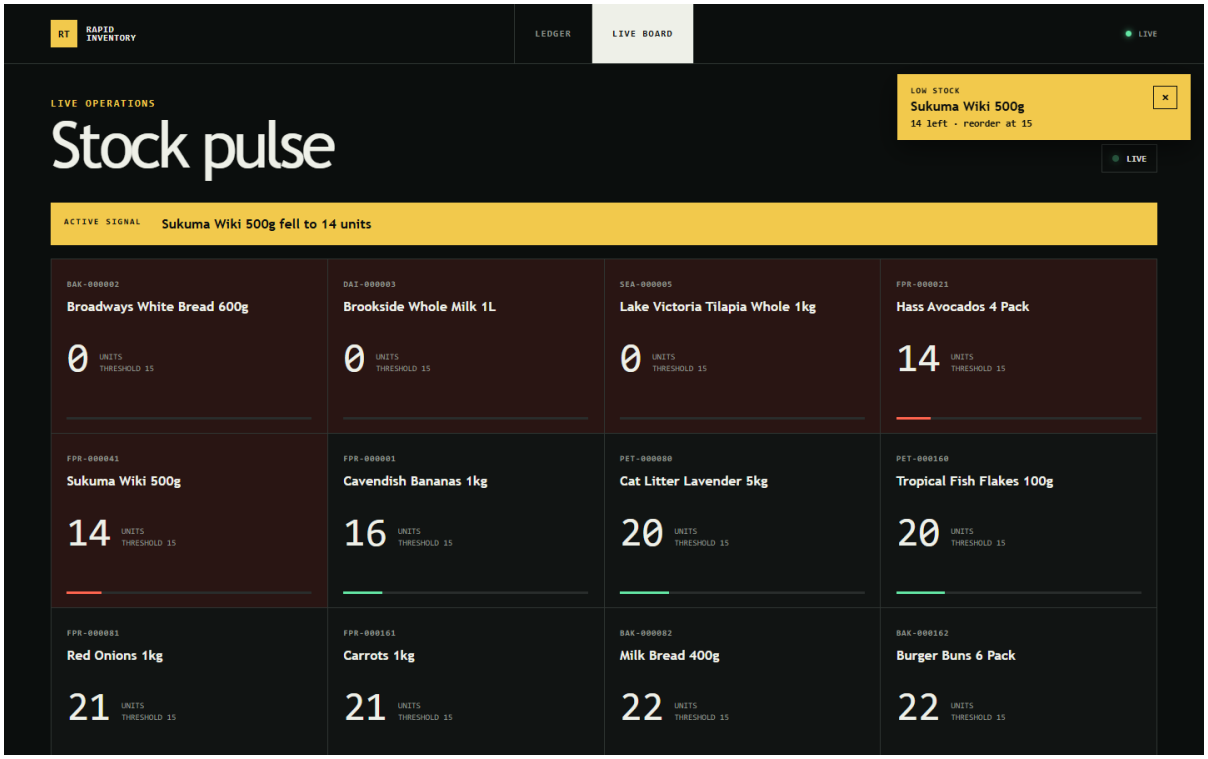


Figure EV-F4 — Live board in Browser B lists the alert

Evidence ID	**EV-F4**
Related issue / test	INV-005 · TC-54
Evidence type	Screenshot (Browser B, Live board)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:38 UTC
Artifact	`screenshots/EV-F4_browserB_live_board_alert.png`
What it demonstrates	Client-side navigation to the Live board in the same session.
Observation	Alert listed in the board's signal bar and as a toast; no duplicate.
Result / status	**PASS**

EV-F5 Actual SignalR frames received by Browser B


```
"at": "2026-10-29T03:18:33.163Z",
"target": "StockUpdated",
"arguments": [
41,
16
"at": "2026-10-29T03:18:33.451Z",
"target": "StockUpdated",
"arguments": [
41,
15
"at": "2026-10-29T03:18:34.849Z",
"target": "StockUpdated",
"arguments": [
41,
14
"at": "2026-10-29T03:18:34.849Z",
"target": "LowStockAlert",
"arguments": [
"productId": 41,
"productName": "Sukuma Wiki 500g",
"stock": 14,
"reorderThreshold": 15
```


Log excerpt EV-F5 — Actual SignalR frames received by Browser B

Evidence ID	**EV-F5**
Related issue / test	INV-005 · TC-53; INV-003 · TC-31, TC-32
Evidence type	WebSocket frames (Chrome DevTools Protocol Network.webSocketFrameReceived)
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 03:18:33–03:18:37 UTC
Artifact	`screenshots/EV-F5_signalr_frames_browserB.json`
What it demonstrates	Hub invocations as received on the wire by Browser B during EV-D/EV-F.
Observation	StockUpdated 41→16, 15, 14, then LowStockAlert {productId 41, "Sukuma Wiki 500g", stock 14, reorderThreshold 15}. No frame for 15 (exact threshold) and none for the failed order.
Result / status	**PASS**

EV-F6 Live low-stock check incl. navigation re-initialisation

```
02:54:48.733 Product FPR-000021 "Hass Avocados 4 Pack" stock 17, threshold 15
02:54:49.417 Browser A: connection live, shows stock 17
02:54:49.419 Browser B: connection live, shows stock 17
02:54:49.425 Browser A: clicked Order (qty 1), expecting stock 16
02:54:49.548 Browser A: shows stock 16
02:54:49.550 Browser B: shows stock 16 without reload  -> PASS
02:54:49.674 Browser A: failed order shows "Order could not be placed."
02:54:51.186 Browser B after failed order: 16 (before 16, database 16)  -> PASS
02:54:52.318 Order to exactly the threshold (15): toasts A=0 B=0, expected none  -> PASS
02:54:52.442 Browser A toast after falling to 14: ["Low stock | Hass Avocados 4 Pack | 14 left · reorder at 15"]  -...
02:54:52.443 Browser B toast after falling to 14: ["Low stock | Hass Avocados 4 Pack | 14 left · reorder at 15"]  -...
02:54:53.953 Failed order: toasts A=1 B=1, expected 1 each  -> PASS
02:54:54.686 Browser B toasts after navigating away and back: 1, expected 1  -> PASS
```

Log excerpt EV-F6 — Live low-stock check incl. navigation re-initialisation

Evidence ID	**EV-F6**
Related issue / test	INV-005 · TC-51…TC-58
Evidence type	Headless two-browser check output
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026 02:54 UTC
Artifact	`logs/EV-F6_two_browser_low_stock_check.txt`
What it demonstrates	seed/two-browser-check.mjs on the final build.
Observation	All 7 checks PASS, including 1 toast after navigating Browser B away and back.
Result / status	**PASS**

5.7 Evidence G — Final Validation

EV-G1 Builds and automated suites

```
$ dotnet build Inventory.sln
Build succeeded.
0 Warning(s)
0 Error(s)



$ dotnet test Inventory.sln
Total tests: 42
Passed: 42


$ npx ng test --watch=false --browsers=ChromeHeadless
TOTAL: 17 SUCCESS


$ npm run build
  | Initial total | 388.10 kB | ...
Application bundle generation complete. [4.206 seconds] - 2026-10-29T02:58:02.779Z
Output location: E:\ALEX\recv\stage-3-assessment\stage-3-assessment\frontend\dist\frontend
```

Log excerpt EV-G1 — Builds and automated suites

Evidence ID	**EV-G1**
Related issue / test	TC-70…TC-73
Evidence type	Build and test logs
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`logs/EV-G1_backend_build.txt; EV-G2_frontend_production_build.log; EV-G3_backend_tests_FINAL_42_passed.log; EV-G4_frontend_karma_tests.log`
What it demonstrates	Final regression run on HEAD 4555d20.
Observation	Backend build 0 warnings / 0 errors; 42/42 backend tests; 17/17 Karma specs; production bundle 388.10 kB.
Result / status	**PASS**

EV-G5 Git history and repository status

```
$ git log --oneline --decorate
$ git status --short (before adding release documents)
?? release/
$ git rev-parse HEAD origin/main
$ git log --format='%h %ad %an %s' --date=iso
4555d20 2026-10-29 06:02:59 +0300 alexkemboi docs: document root causes, fixes and verification
280ac2d 2026-10-29 05:58:13 +0300 alexkemboi test: add dependency-free load generator for multi-instance order checks
1bfb08b 2026-10-29 05:55:05 +0300 alexkemboi feat: show low stock alerts as global toasts
75569d9 2026-10-29 05:51:04 +0300 alexkemboi refactor: extract order placement into OrderService
d8329cc 2026-10-29 05:48:34 +0300 alexkemboi fix: broadcast stock updates to all watching clients
2f076ee 2026-10-29 05:45:44 +0300 alexkemboi perf: remove N+1 category lookups from inventory list
6157517 2026-10-29 05:43:03 +0300 alexkemboi fix: prevent concurrent stock overselling
6028cbd 2026-10-29 05:40:03 +0300 alexkemboi test: make last-unit race deterministic across queued lock waiters
6bae192 2026-10-29 05:31:38 +0300 alexkemboi test: add SQL Server integration tests reproducing order, list and Sig...
fc3c873 2026-10-29 05:07:49 +0300 alexkemboi chore: baseline snapshot of provided project (no prior git history pre...
```

Log excerpt EV-G5 — Git history and repository status

Evidence ID	**EV-G5**
Related issue / test	Traceability
Evidence type	git log / status
Environment	Local Docker Compose stack (api, web, db) on Windows 11 host; SQL Server 2022 (16.0.4305.5)
Captured	29 Oct 2026
Artifact	`logs/EV-G5_git_history_and_status.txt`
What it demonstrates	Incremental commits on main, pushed to origin; no history rewritten.
Observation	HEAD = origin/main = 4555d20 at capture; only the new release/ folder was untracked.
Result / status	**PASS**

5.8 Evidence Traceability Matrix

Issue	Root cause	Code change (commit / file)	Tests	Evidence	Observed result	Status
INV-001	Lost update; no transaction	6157517 → 75569d9 · OrderService.PlaceOrderAsync; AppDbContext; migration	TC-01…TC-14	EV-B1…B4, EV-A2	No oversell; stock ≥ 0; rollback	PASS
INV-002	N+1 per-row category query	2f076ee · ProductsController.List	TC-20…TC-25	EV-C1…C3	202 → 2 cmds; 224.9 → 11.9 ms	PASS
INV-003	Update sent to requester only	d8329cc · IStockNotifier.cs; stock-realtime.service.ts	TC-30…TC-36	EV-D0…D6, EV-F5	B updates without reload	PASS
INV-004	Logic in controller; stale duplicate service	75569d9 · OrderService; OrdersController	TC-40…TC-45	EV-E1…E3	13 contracts unchanged	PASS
INV-005	Alert UI only on board; key collisions	1bfb08b · low-stock-alerts.ts; stock-realtime.service.ts	TC-50…TC-59	EV-F1…F6	Toast in all browsers; correct threshold	PASS

6. Installation Guide and Environment Setup

6.1 Prerequisites

Requirement	Version / detail	Source
Docker Desktop + Docker Compose	≥ 6 GB memory for Docker; verified with Docker 29.8.2 / Compose v5.6.0	README; test host
Free ports	1433 (SQL Server), 4200 (web), 5080 (API)	README / docker-compose.yml
Container images	mcr.microsoft.com/mssql/server:2022-latest; mcr.microsoft.com/dotnet/sdk:9.0; node:24-alpine	docker-compose.yml, Dockerfiles
For running tests on a host	.NET SDK 9 (or later SDK with .NET 9 runtime), Node 20+/24, Chrome (Karma)	Test project / package.json
Network	First start downloads ≈ 3 GB of images; seeding 250,000 orders adds time	Observed

6.2 Configuration and Environment Variables

Variable	Service	Purpose
`MSSQL_SA_PASSWORD`, `ACCEPT_EULA`	db	SQL Server administrator password (development value in docker-compose.yml — must be replaced by a secret in any shared environment)
`ConnectionStrings__DefaultConnection`	api / tests	SQL Server connection string (do not commit production credentials)
`ASPNETCORE_ENVIRONMENT`, `ASPNETCORE_URLS`	api	Development; listens on 0.0.0.0:8080 (published as 5080)
`SEED_ORDER_COUNT`	api	Historical orders to seed when the database is empty (250000)

Credentials are intentionally not reproduced in this document.

6.3 Installation Steps

- Obtain the release: clone `main` at the release commit or extract `rapid-inventory-1.1.0-src-4555d20.zip` and verify CRC32 `CE4AA374` / SHA-256 (Section 2.6).
- Database setup: no manual step. On API start, EF Core applies pending migrations (`InitialCreate`, `AddNonNegativeStockConstraint`) and seeds 20 categories, 2,000 products and 250,000 orders if the database is empty. For an existing database, take a backup first (Section 7).
- Environment setup: from the repository root run `docker compose up --build`. Wait until `docker compose ps` shows `db` as healthy and the API logs "Application started".
- Optional reset of all data: `./seed/reset.sh` (runs `docker compose down -v` — destroys the database volume).

6.4 Post-deployment Verification (Test Instruction)

Check	How	Expected	Verified in test env
SQL Server	`docker compose ps`	db: Up (healthy)	**PASS** EV-A2
Migrations	Query `__EFMigrationsHistory`; `sys.check_constraints`	Both migrations; CK_Products_Stock_NonNegative	**PASS** EV-A2
API	http://localhost:5080/health	{"status":"healthy"}	**PASS** EV-A2
Web app	http://localhost:4200	Inventory ledger loads; header status LIVE	**PASS** EV-D1
SignalR	Two browsers; order in one	Other browser updates without refresh	**PASS** EV-D4
Integrity	`./seed/stress-orders.sh` then compare stock and order lines	Units sold = stock deducted; stock ≥ 0	**PASS** EV-B4

Known environment issue: on hosts where a local SQL Server (e.g. SQL Express) already listens on 1433, host tools connecting to `localhost,1433` reach the local instance instead of the container (containers are unaffected). During testing the container was additionally published on 14333 through an uncommitted Compose override.

7. Release Prerequisites

#	Prerequisite	Status	Note
1	Mandatory automated regression tests pass	PASS	Backend 42/42, frontend 17/17, builds clean (test environment)
2	Independent QA review of this report and re-execution	Outstanding	QA reviewer to be named
3	Deployment to a staging / production-like environment and smoke test	Outstanding	Only local Docker Compose tested
4	Database backup taken and restore tested before applying migration	Outstanding	No backup procedure evidenced
5	Migration `AddNonNegativeStockConstraint` pre-check: no existing row with Stock < 0	Outstanding for target DB	Verified 0 rows in test DB (EV-A2); migration fails if any exist
6	Environment configuration reviewed (secrets, connection strings, CORS origin, logging level)	Outstanding	Compose uses development values
7	Approved release artifact produced and checksummed	Partial	Source zip + CRC32/SHA-256 computed; not yet approved/signed
8	Known issues and residual risks accepted (Section 9)	Outstanding	Business/technical owner decision
9	If more than one API instance: SignalR backplane configured	Outstanding / conditional	Not implemented
10	Rollback procedure reviewed and rehearsed	Outstanding	Section 8
11	Authorised deployment approval	Outstanding	Section 10 sign-off pending

8. Rollback Procedure

Architecture: stateless API and Angular containers built from source, one SQL Server database. The only schema change is an added CHECK constraint; no columns or data were changed, so the previous application version runs correctly against the new schema.

- **Decide and freeze.** Stop new deployments; record the time and reason for rollback.
- **Restore the previous application version.** Check out the previous release commit (baseline `fc3c873` or the previously deployed tag) and rebuild/redeploy the `api` and `web` images together (`docker compose up --build -d api web`). Backend and frontend must be rolled back as a pair: the new frontend expects group broadcasts but remains compatible with the old API; the old frontend works with the new API.
- **Database: do not restore a backup to undo this release.** Restoring would discard valid orders and stock movements made since deployment. Keep the database as is — the previous version is compatible with the added constraint.
- **Optional schema rollback.** Only if required, drop the constraint with the migration's Down step (`dotnet ef database update 20261023081746_InitialCreate`, which runs `DropCheckConstraint`). This removes no data. Note: without the constraint and with the old code, overselling can recur.
- **Configuration.** Restore the previous environment variables / compose file from version control.
- **Post-rollback health checks.** `docker compose ps` (db healthy), `GET /health`, web app loads, place and cancel one test order.
- **Inventory integrity check.** Run: `SELECT COUNT(*) FROM Products WHERE Stock < 0` (expect 0) and compare recent order-line totals with stock movements for affected products; investigate any discrepancy before reopening.

Limitation: rollback has not been rehearsed, and no backup/restore procedure exists in the repository. A tested backup and a rollback rehearsal in staging are recommended before production release.

9. Known Issues and Residual Risks

#	Item	Impact	Recommendation
1	No SignalR backplane	With >1 API instance, users miss updates/alerts from other instances	Redis backplane or Azure SignalR before scaling out
2	At-most-once notifications after commit	An alert can be lost if publishing fails (logged)	Transactional outbox if delivery must be guaranteed
3	No idempotency key	Client retry after a timeout can create a duplicate order	Idempotency-Key header + unique index
4	Multi-line deadlock possible	Rare 500 with full rollback (not observed)	Lock products in id order or retry on error 1205
5	Hot-product contention	Measured: doubling concurrency on one product kept throughput flat (135 → 121 req/s) and raised p50 140 → 326 ms	Shorter transactions; measure Release build
6	Alerts not persisted	Users connecting later do not see earlier alerts	Persist or derive from current stock if required
7	Toast overlaps heading counter	Cosmetic (EV-F2)	Offset toast region below page heading
8	PUT /api/products overwrites stock	Admin edit can overwrite concurrent deductions	Require RowVersion from client
9	Walkthrough video not recorded	Assessment deliverable outstanding	Record using docs/WALKTHROUGH_SCRIPT.md
10	Brand fonts not embedded	Manrope/Inter (website) were not installed; Segoe UI used	Install brand fonts and refresh styles if required

10. Release Acceptance and Sign-off

Role	Name	Decision	Signature	Date
Developer / Release Maker	[To be confirmed]	[Pending]	[Pending]	[Pending]
QA Reviewer	[To be confirmed]	[Pending]	[Pending]	[Pending]
Technical Approver	[To be confirmed]	[Pending]	[Pending]	[Pending]
Business Owner	[To be confirmed]	[Pending]	[Pending]	[Pending]

Release recommendation	
Recommendation	**Ready subject to outstanding conditions**
Justification	All five reported issues are resolved and verified in the test environment: the 9 baseline defect reproductions now pass, 42/42 backend and 17/17 frontend tests pass, builds are clean, and live browser and multi-process checks confirm the behaviour. No test is FAIL or BLOCKED.
Conditions before production	Independent QA review; staging deployment and smoke test; database backup and migration pre-check; configuration/secret review; acceptance of residual risks (Section 9), including a SignalR backplane if more than one API instance will run; rollback rehearsal; authorised approval (table above).
Production readiness	**Not yet established.** This document does not authorise deployment; no approval has been given.